Lab 1
Information Flow Control

Ana-Maria Dobrescu, Aurora Navas Jimenez

April 17, 2026

1 Introduction

In this document we will explain how we solved the lab. First we will start by explaining how our server works and then move on to the clients. We implemented two benign clients, Peter and MJ, and two malicious clients, Green Goblin, who attempts to count how many clients are in the database, and Dr. Echo, who instead of sending its own profile when matching it echoes back the other person's profile. By doing this, our malicious client will gain access to the confidential information, without sending anything in return.

2 Server

Our server contains of six functions: `server`, `loop`, `main`, `sendMatch`, `tryMatchOne` and `tryMatchAgainstAll`. We will explain each function separately.

2.1 main and server

Our `server` function contains all the the other helper function whereas our `main` function follows the same structure as the given one, the only modification being that we are also calling our server, not just the dispatcher.

2.2 loop

The `loop` function receives the list of profiles as input. By running this function recursively, we can append new entries to the list so that we can continously check if the already existing profiles match with the new ones. Whenever a new profile is entered it will call `tryMatchAgainstAll` to see if we can have any matches.

2.3 tryMatchAgainstAll

The `tryMatchAgainstAll` function takes a new profile and list of existing profiles to check for potential matches. It runs recursively and applies `tryMatchOne` for two profiles to check for their match.

2.4 tryMatchOne

We consider this function to be the most important function of our server as this is where the actual matching happens and also part of the declassification. The function takes to profiles as arguments and then extracts the information from the profile. It executes each client's agent function on the other client's profile and decides if they want to match. The resulting outputs are sensitive, so the server will declassify the matching outcome down to the public level. If both profile agents ended with a match, the server sends the NEWMATCH messages to the two clients in separate processes. If the answer is negative, the server will simply skip the send statement

2.5 sendMatch

This function is simply in charge of sending the NEWMATCH message to the two clients.

3 Benign clients

Our benign clients, MJ and Peter, use a really similar dating service protocol, only differing in the matching criteria, while properly following information flow control. Both of them have four functions: `client`, `sendToServer`, `myAgent`, and `waitForMatches`. We will explain each function separately.

3.1 client

In the benign clients, `client` is our main function. It first defines the client's profile properties (name, birth year, gender, interests) and raises them to MJ's level. Then, it creates the profile itself and sends it using the `sendToServer` function. In the end, it runs the `waitForMatches` function.

3.2 sendToServer

This function's purpose is to send the profile to the server. To comply with the server's needs, it sends the profile, the executable discovery agent function, and the client's process ID.

3.3 myAgent

`myAgent`'s input is the other client's profile. To make this explanation clearer, we will use a practical example using MJ and Peter. We will assume the agent that is being run is MJ's.

First, the function takes Peter's profile to compare it against MJ's interests. It also defines MJ's profile, locking them at her own level. Then, `myAgent` evaluates the match conditions. In MJ's case, she wants the other client's interests to be music and gender to be male. In Peter's case, he wants the other person's gender to be female.

If the match happens, MJ sends her profile back to the server along with a true boolean and declassifies it to Peter's level. If the match does not happen, it sends back a false boolean and an empty tuple.

3.4 waitForMatches

It acts like a loop. Like the name states, it waits to receive a "NEWMATCH" string and a profile, and prints the match's profile.

4 Malicious clients

4.1 Green Goblin

Green Goblin's purpose is to know how many clients are using this dating server. It holds the same functions as the benign clients; in fact, `client` and `sendToServer` are exactly the same.

In the Goblin's `myAgent` function, he sends the string "PROFILE" to himself every time the function is run, which he then would count to know the number of users. Everything else in the agent is straightforward and identical in concept to the benign clients. He sends his profile to his match and declassifies it to the match's level. We have decided he matches with everyone, but it is completely irrelevant for his purpose.

The `waitForMatches` function has been changed to `waitForAgentExecution`. In it, it prints the match's profile if there is one, like in `waitForMatches`. But additionally, it waits for the string "PROFILE" to be sent by itself, and it prints "NEW PROFILE FOUND" if it receives it. By counting all the "NEW PROFILE FOUND", Goblin can count the number of users.

4.2 Dr. Echo

Dr. Echo gets access to his match's profile, but instead of sending his profile to his match he will echo their profile, thus getting access to sensitive information without sending anything in return. The overall logic of this client is the same as for the benign clients, the difference being in the agent. Instead of having a preference for matching, it will accept everyone and instead of releasing their profile, he will just steal the other person's profile and echo it back.

5 Contributions

Both members of the team have always met in person to work on the project together. The only thing that can be attributed to us individually is the programming of the malicious clients. Aurora did the Green Goblin, and Ana did Mr. Echo.